

Modifications Backend Requises

Projet : Holy Fork -- Systeme de gestion utilisateurs, roles & permissions **Date** : 2026-07-17 **Destinataire** : Developpeur backend Django **Priorite** : P0 a P2

Contexte

Le frontend a implemente un systeme d'invitation d'employes avec creation de compte, une matrice de permissions (lecture seule), et des guards UI par permission. Plusieurs endpoints backend doivent etre crees ou modifies pour que le systeme fonctionne de bout en bout.

P0 -- Correctifs critiques (fonctionnalite cassee)

1. POST /api/settings/users/ -- Verifier les permissions

Fichier : `apps/settings/views.py` (lignes 86-104)

Probleme actuel : L'endpoint utilise `UserRegistrationSerializer` (le meme que `/api/auth/register/`). Il n'y a pas de verification de permission -- n'importe quel user authentifie peut creer des comptes.

Correction requise :

- Ajouter `permission_classes = [HasPermission]` avec `required_permission = Permission.MANAGE_STAFF`
- S'assurer que seul un admin/gerant peut creer des comptes via cet endpoint
- L'endpoint public `/api/auth/register/` reste sans permission (auto-inscription)

```
# apps/settings/views.py
from apps.staff.permissions import HasPermission
from apps.staff.employee_roles import Permission

class UsersSettingsView(APIView):
    permission_classes = [IsAuthenticated, HasPermission]
    required_permission = Permission.MANAGE_STAFF
    # ... reste du code
```

2. POST /api/settings/users/ -- restaurant_id optionnel

Probleme actuel : `UserRegistrationSerializer` exige `restaurant_id` (required). Quand un admin cree un employe, le `restaurant_id` est fourni par le frontend (depuis le formulaire "Etablissement"). Mais si l'admin veut creer un employe sans l'assigner immediatement, ca echoue.

Correction requise :

- Rendre `restaurant_id` optionnel dans `UserRegistrationSerializer`
- Si `restaurant_id` est fourni, creer le `RestaurantEmploye`
- Si absent, creer juste le `HollyUser` + `Employe` sans assignment

```
# apps/authentication/serializers.py - UserRegistrationSerializer
restaurant_id = serializers.IntegerField(required=False, allow_null=True)
```

```
def create(self, validated_data):
    # ... creer user + employe
    restaurant_id = validated_data.get('restaurant_id')
    if restaurant_id:
        RestaurantEmploye.objects.create(
            restaurant_id=restaurant_id,
            employe=employe
        )
    return user
```

P1 -- Fonctionnalités manquantes (systeme de roles)

3. PATCH /api/employees/{id}/ -- Changement de role

Probleme actuel : L'endpoint accepte `type_employe_id` dans le body, ce qui permet techniquement de changer le role. Mais il n'y a pas de validation :

- Un serveur pourrait se promouvoir gerant
- Pas de verification de hierarchie

Correction requise :

- Ajouter une permission `MANAGE_STAFF` pour modifier `type_employe_id`
- Verifier que le user qui fait la modification a un role hierarchiquement superieur au nouveau role assigne
- Utiliser `PermissionChecker.is_higher_than()` deja implemente dans `apps/staff/permissions_utils.py`

```
# Dans le serializer ou la view employes
def update(self, instance, validated_data):
    new_type_id = validated_data.get('type_employe_id')
    if new_type_id and new_type_id != instance.type_employe_id:
        checker = PermissionChecker(user=self.context['request'].user)
        new_type = TypeEmploye.objects.get(id=new_type_id)
        new_role = EmployeeRole(new_type.nom_type)
        if not checker.is_higher_than(new_role):
            raise PermissionDenied("Impossible d'assigner un role superieur au votre")
    return super().update(instance, validated_data)
```

4. GET /api/settings/users/ -- Filtrer par restaurant

Probleme actuel : L'endpoint retourne TOUS les users du systeme, sans filtrage par restaurant. Le parametre `restaurant_id` est ignore.

Correction requise :

- Filtrer les users par restaurant via la jointure `RestaurantEmploye`
- N'afficher que les users lies aux restaurants du demandeur

```
def get(self, request):
    restaurant_id = request.query_params.get('restaurant_id')
    if restaurant_id:
        employe_ids = RestaurantEmploye.objects.filter(
            restaurant_id=restaurant_id
        ).values_list('employe__user_id', flat=True)
        users = HollyUser.objects.filter(id__in=employe_ids)
    else:
        # Filtrer par restaurants accessibles au demandeur
```

```

checker = PermissionChecker(user=request.user)
restaurant_ids = checker.get_restaurants()
employee_ids = RestaurantEmployee.objects.filter(
    restaurant_id__in=restaurant_ids
).values_list('employee__user_id', flat=True)
users = HollyUser.objects.filter(id__in=employee_ids)
# ...

```

5. PATCH /api/settings/users/{id}/ -- Desactiver un compte

Probleme actuel : L'endpoint `PatchedUserProfileRequest` n'accepte que `email`, `first_name`, `last_name`. Impossible de desactiver un compte.

Correction requise :

- Ajouter `is_active` au serializer (écriture reservee aux `MANAGE_STAFF`)
- Quand `is_active=False`, le user ne peut plus se connecter

```

# apps/settings/serializers.py ou apps/authentication/serializers.py
class AdminUserUpdateSerializer(serializers.ModelSerializer):
    class Meta:
        model = HollyUser
        fields = ['email', 'first_name', 'last_name', 'is_active']

```

P2 -- Envoi d'emails d'invitation

6. Configurer Django email backend

Probleme actuel : Aucun systeme d'envoi d'email n'est configure. Le modele `NotificationSettings` stocke des toggles mais rien n'est envoye.

Correction requise :

6a. Configuration SMTP dans `settings.py` :

```

# holly_pi/settings.py
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = os.environ.get('EMAIL_HOST', 'smtp.sendgrid.net')
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.environ.get('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = os.environ.get('EMAIL_HOST_PASSWORD')
DEFAULT_FROM_EMAIL = 'Holy Fork <noreply@hollyfork.fr>'

```

6b. Template d'email d'invitation : Creer `apps/authentication/templates/emails/invitation.html` avec :

- Nom du restaurant
- Email de connexion
- Mot de passe temporaire
- Lien vers le dashboard
- Message "Changez votre mot de passe a la premiere connexion"

6c. Envoi dans le flow de creation :

```
# apps/authentication/serializers.py - dans create()
from django.core.mail import send_mail
from django.template.loader import render_to_string

def create(self, validated_data):
    user = # ... creation du user

    # Envoyer l'email d'invitation si c'est une creation admin
    if self.context.get('is_admin_invite'):
        password = validated_data['password']
        html = render_to_string('emails/invitation.html', {
            'name': user.get_full_name(),
            'email': user.email,
            'password': password,
            'login_url': settings.FRONTEND_URL + '/login',
        })
        send_mail(
            subject='Bienvenue sur Holy Fork',
            message='',
            html_message=html,
            from_email=settings.DEFAULT_FROM_EMAIL,
            recipient_list=[user.email],
        )
    return user
```

7. POST /api/auth/forgot-password/ -- Reset mot de passe

Probleme actuel : Aucun endpoint de reset password n'existe. Les employes invites avec un mot de passe temporaire ne peuvent pas le changer.

Correction requise :

7a. Endpoint demande de reset :

```
POST /api/auth/forgot-password/
Body: { "email": "user@example.com" }
Response: { "message": "Email envoye si le compte existe" }
```

7b. Endpoint confirmation de reset :

```
POST /api/auth/reset-password/
Body: { "token": "xxx", "password": "new_pass", "password2": "new_pass" }
Response: { "message": "Mot de passe modifie" }
```

7c. Endpoint changement de mot de passe (user connecte) :

```
POST /api/auth/change-password/
Body: { "old_password": "xxx", "new_password": "yyy", "new_password2": "yyy" }
Response: { "message": "Mot de passe modifie" }
```

P2 -- Ameliorations (non bloquantes)

8. Reponse GET /api/employees/ -- Inclure l'email du user lie

Probleme actuel : La reponse `Employee` contient `user_id` mais pas l'email du user lie. Le frontend ne peut pas afficher l'email d'un employe.

Correction requise :

- Ajouter un champ `email` (read-only) dans `EmployeeSerializer`

```
class EmployeeSerializer(serializers.ModelSerializer):
    email = serializers.SerializerMethodField()

    def get_email(self, obj):
        return obj.user.email if obj.user else None

    class Meta:
        model = Employee
        fields = [..., 'email']
```

9. Endpoint pour lister les roles disponibles avec descriptions

Probleme actuel : `GET /api/staff/permissions/roles/` retourne les roles mais pas de description lisible. Le frontend hardcode les labels.

Correction requise :

- Ajouter un champ `description` a chaque role dans la reponse
- Sourcer depuis `TypeEmployee.description` ou depuis un mapping dans `employee_roles.py`

Resume des endpoints a modifier/creer

Endpoint	Action	Priorite
POST /api/settings/users/	Ajouter permission <code>MANAGE_STAFF</code>	P0
POST /api/settings/users/	Rendre <code>restaurant_id</code> optionnel	P0
PATCH /api/employees/{id}/	Valider hierarchie sur changement de role	P1
GET /api/settings/users/	Filtrer par restaurant	P1
PATCH /api/settings/users/{id}/	Ajouter <code>is_active</code>	P1
Config Django	Configurer email backend (SMTP/SendGrid)	P2
POST /api/auth/forgot-password/	Creer endpoint reset password	P2
POST /api/auth/reset-password/	Creer endpoint confirmation reset	P2
POST /api/auth/change-password/	Creer endpoint changement password	P2
GET /api/employees/	Ajouter <code>email</code> dans la reponse	P2

Endpoint	Action	Priorite
GET /api/staff/permissions/roles/	Ajouter descriptions	P2

Fichiers backend concernes

Fichier	Modifications
apps/settings/views.py	Permissions sur UsersSettingsView, filtrage par restaurant
apps/authentication/serializers.py	restaurant_id optionnel, envoi email
apps/staff/views.py ou serializer employes	Validation hierarchie sur changement de role
apps/authentication/views.py	Endpoints forgot-password, change-password
holly_pi/settings.py	Configuration EMAIL_BACKEND
apps/authentication/templates/	Templates email (a creer)